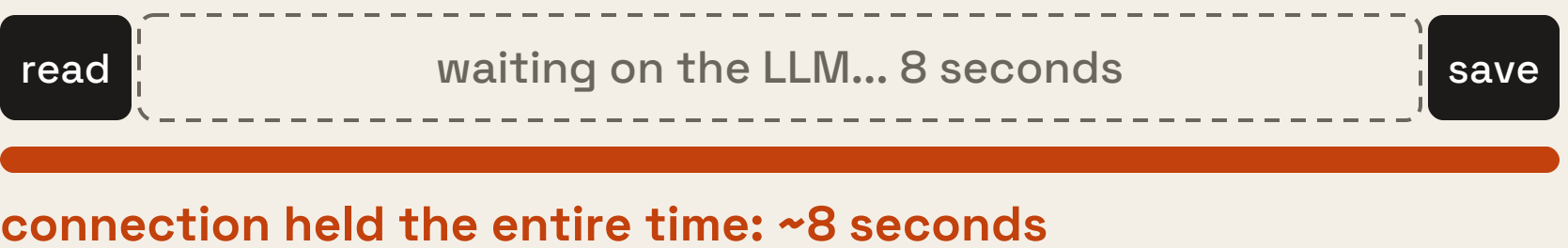


a 2-minute read, promise

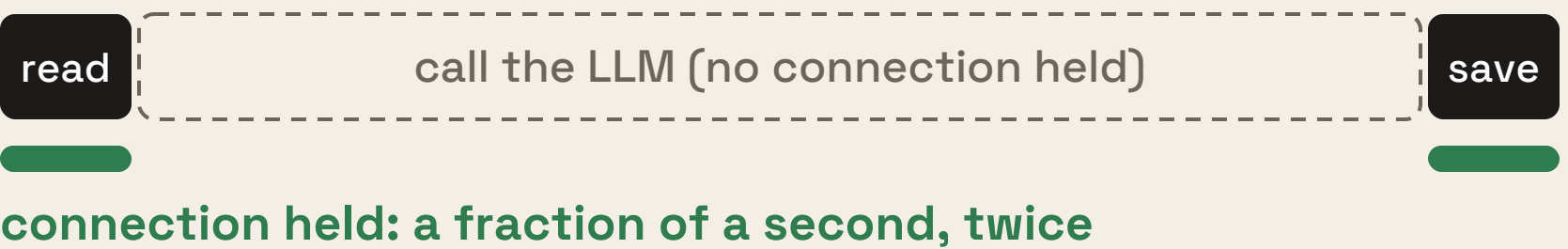
Why your API dies while your database naps

Your endpoint grabs a database connection, then calls an LLM (or Stripe, or any slow API) and waits. The whole time, that connection just sits there, held.

What most code does



What it should do



Think of a restaurant with 10 tables. Every guest sits down and waits for a delivery from somewhere else. The kitchen is empty. The line outside is out the door.

10 connections, each held 8s = about 1.25 requests/sec. Total.

Is this you?

- ☐ You call an API while a database transaction is open.
- ☐ Your API times out, but database CPU looks fine.
- ☐ You never changed the pool size (often just 10).

Two ticks? Keep reading.

Catch it in the act

Run this at peak traffic. Each row is a connection stuck in an open transaction, doing nothing:

```
SELECT pid, application_name,
       now() - state_change AS idle_for,
       now() - xact_start   AS txn_open_for,
       left(query, 80)      AS last_query
FROM pg_stat_activity
WHERE state = 'idle in transaction'
ORDER BY idle_for DESC;
```

Look at `idle_for`. Seconds, not milliseconds? Something's waiting on an API. No rows doesn't mean you're safe. It's a snapshot, so run it a few times. (That's Postgres. Other databases have something similar.)

Fix it

- 1 Make the slow call outside the transaction.**
Read, release the connection, call the API, then save.
- 2 Don't let requests wait forever for a connection.**
Set a short pool timeout. A quick error beats a hang.
- 3 Watch how long requests wait for a connection.**
It spikes even while database CPU looks perfectly fine.

Useful? I break down one backend mistake like this every day.
[@gouravkargwal](#)